

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/365431504>

Real-time monitoring for manual operations with machine vision in smart manufacturing

Article in *Journal of Manufacturing Systems* · October 2022

DOI: 10.1016/j.jmsy.2022.10.015

CITATIONS

25

READS

314

5 authors, including:

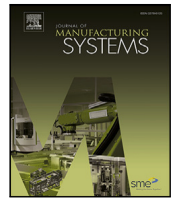


[Yuhang Zeng](#)

Wuhan University of Technology

5 PUBLICATIONS 32 CITATIONS

SEE PROFILE



Real-time monitoring for manual operations with machine vision in smart manufacturing

Ping Lou^a, Ji Li^a, YuHang Zeng^a, Bing Chen^{b,*}, Xiaomei Zhang^a

^a School of Information Engineering, Wuhan University of Technology, Wuhan, Hubei, 430070, China

^b School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, Hubei, 430074, China

ARTICLE INFO

Keywords:

Manual action recognition
Real-time operation monitoring
Deep learning
Repetition counting
Object detection

ABSTRACT

Online real-time production process monitoring is the basis for intelligent manufacturing refinement management. This paper proposes a contactless monitoring framework with machine vision to realize real-time production/assembly process monitoring and repetition counting. In addition, a two-stage approach is presented to solve the issue of manual operation recognition and counting by pieces. In the first stage, a high-efficient method for object detection and classification, YOLOv4-tiny, is used to recognize manual operations/sub-operations and classify the sub-operations of the operations using monitoring video. And a Sliding Window Counter algorithm is utilized to find appropriate boundary points of different categories of the sub-operations for counting in the second stage. A dataset of repetitive manual operations is constructed for training this approach. The experiment results on the production/assembly line operations indicate that the approach can identify and count worker operations in real time.

1. Introduction

With the rapid development of the Industrial Internet of Things (IIoT) and artificial intelligence, as well as their applications in intelligent manufacturing, it is possible to monitor production conditions and efficiently elaborately manage productions [1–3]. However, the conditions and workpiece ratio of production is tough to be monitored in real-time in human-operated production/assembly line due to the lack of effective means to monitor human operations and piece counting. The manual production/assembly lines' workpiece ratios are usually ex post facto statistics and cannot count in real-time. Hence it is tough for these lines to achieve efficient and intelligent management. Nevertheless, the recent advances in machine learning and machine vision create new opportunities for intelligent and efficient manufacturing systems [4–7]. With deep learning and machine vision, Chen et al. [8] monitor the assembly process of workers and recognize the worker's actions with pose estimation [9] to prevent assembly quality issues. Xiong et al. [10] use a dual-stream Convolutional Neural Network (CNN) for recognizing manual action, and the approach can improve operational efficiency. Tao et al. [11] use Inertial Measurement Unit (IMU) and surface Electromyography (sEMG) signals with CNN to identify worker activities, as well as quantify and evaluate worker performance. It seems possible to monitor and recognize the worker's action in production/assembly lines with deep learning and machine

vision. However, it is still in its infancy and faces many challenges for complex work operations.

Recently repetitive action counting approaches with deep learning and machine vision have been studied, and some end-to-end approaches are presented to mainly used for the repeated counting of the periodic actions [12–14]. Usually, these actions are composed of one-action, and their cycles are shorter, always less than 5 s (namely short-period actions) [15,16]. However, workers often need more than one-action to complete an operation in the production/assembly lines. Each operation is always composed of several one-actions, and its cycle is always relatively longer (namely long-period actions) and has variations in temporal duration. Obviously, the repetition counting of long-period actions is more complex and challenging.

However, these related researches have provided valuable insights:

(1) The feature encoding for repeated actions is Spatio-temporal self-similarities.

(2) Considering the Spatio-temporal characteristics, long-range temporal dependencies are essential to encode.

(3) The feature encoding of actions can be represented as an intermediate layer for further processing.

Hence, the real-time monitoring and counting framework is presented for monitoring a specific manual operation and counting by pieces in a production/assembly line. And a two-stage approach is

* Corresponding author.

E-mail address: chenbing@hust.edu.cn (B. Chen).

<https://doi.org/10.1016/j.jmsy.2022.10.015>

Received 6 April 2022; Received in revised form 12 October 2022; Accepted 21 October 2022

Available online 15 November 2022

0278-6125/© 2022 The Society of Manufacturing Engineers. Published by Elsevier Ltd. All rights reserved.

presented for counting by pieces of manual operations. In this two-stage approach, an object detection approach named YOLOv4-tiny is used to recognize and classify workers' operations in the first stage. Then a sliding window algorithm is applied to process the results obtained from the first stage to count by pieces.

The main contributions of this paper are summarized as follows:

- (1) The contactless and online real-time perception method with machine vision is presented to monitor workers' manual operations of a production/assembly line in real time.
- (2) A two-stage approach, involving an object detection in the first stage and a sliding window algorithm for counting in the second stage is put forward to count by pieces for manual operations of a production/assembly line in real-time.
- (3) A dataset of *Repetitive Manual Operations*¹ is constructed and open-sourced. Furthermore, the new data-labelled method is proposed to classify the operation into two sub-operations according to the time distribution of one-actions.

This paper is organized as follows. Section 2 describes the related work on our approach. Section 3 describes the research framework and the algorithms used to tackle the problem. Section 4 describes the dataset construction process and the data's different applications. In Section 5, the method is validated, and the results show that the method can monitor and count by pieces in real time for manual operations of a production/assembly line. The conclusions are made in Section 6.

2. Related work

Current approaches to repetition counting can be broadly classified into two categories. The end-to-end approaches [12,14] use the video or audio of the actions as the input and the counting result directly as the output. And two-stage approaches [9,17] generally use the artificial neural network in the first stage to recognize the input video actions. The second stage processes the output information of the first stage to obtain the counting results. The two-stage approach in this paper applies an object detection approach to obtain action categories in the first stage and then count by pieces in the second stage according to the classification results of the first stage. Selecting and using a suitable object-detection is crucial to guarantee accuracy and efficiency in realizing operation recognition in real-time.

2.1. Repetition counting

Currently, researches on repetition counting mainly focus on simple actions composed of one-action. Typically the counting of simple actions is obtained by Fourier analysis, filtering, peak detection, classification, and singular value decomposition [18–20]. Levy and Wolf [21] proposed a deep neural net for recognizing and classifying. However, these methods cannot handle repetitions with variations in temporal duration. While Runia et al. [22] have proposed non-stationary repetitive action counting algorithms using wavelet filters to convolve the motion maps in time to estimate the dynamics of repetitions.

End-to-end approach. The end-to-end approach directly processes action video to obtain counting results. Zhang et al. [13] have proposed a bidirectional context-aware regression model and constructed a new benchmark *UCFRep*. And a strategy from coarse to fine double-cycle is proposed to estimate cycle lengths for the entire video. Dwibedi et al. [12] collected a new challenging dataset *Countix* including more than 6000 videos with activity repetition counts in the real world. Their method uses temporal self-similarity as an intermediate layer in the end-to-end neural network architecture to cleverly exploit the periodicity of actions. In contrast to the above approach, which focuses

only on visual information, Zhang et al. [14] presented a method that combined video with audio information, which performed well, especially under poor visual conditions.

Two-stage approach. In contrast to the end-to-end approaches, the two-stage approaches involve two phases: recognition and counting. Chen et al. [8] used object detection to identify assembly actions in the first stage, and then pose estimation [9] was used to recognize the human joint and judge the operating times of repetitive assembly actions according to the joint coordinates. Soro et al. [17] used a CNN to perform exercise classification in the recognition stage, then the raw data and previously information detected by the recognition CNN is fed into the repetition counting CNN to get results.

The end-to-end approach shows how to exploit the periodicity of repetitive operations. And the two-stage approach simplifies the complexity of the task by completing sub-tasks in stages. Our research utilizes the above insights to address the problem of identifying and counting long-period and complex manual operations in manufacturing scenarios.

2.2. Deep learning-based object detection

Object detection is an essential part of the recognition operation method in this paper. The fundamental element in the object detection is the Backbone structure for extracting the features of images. In our approach, different one-stage and two-stage object detections are selected according to accuracy and efficiency.

Backbone architectures. Simonyan et al. [23] proposed VGG, which is implemented by stacking a series of smaller convolutional kernels. In the single network performance category, VGG performs very well and soon becomes the most commonly used Backbone in object classification and detection models. Kaiming He et al. [24] used residual learning to stack convolutional layers to mitigate the performance decay. It is achieved by adding a skip connection between layers. Its idea has spawned many other networks. Wang et al. [25] argued that reducing the repetitive gradient information in the network could reduce a massive amount of computation in the inference process. They proposed CSPNet to create different paths for the gradient flow in the network.

One-stage and two-stage object detections. For two-stage detection, such as faster R-CNN [26], generates region proposals by Region Proposal Network (RPN) [27], which leads to the high accuracy of object recognition. For the one-stage detection, such as YOLO [28], the object classification and bounding-box regression are done directly without using pre-generated region proposals, resulting in a faster inference speed than two-stage detections. And Single Shot MultiBox detection (SSD) [29] combines the advantages of the above two methods, using multiple prediction branches to detect objects at different scales, balancing accuracy and speed. For better detection of objects at different scales, EfficientDet [30] is based on EfficientNet [31], which uses multiple stacked feature extraction networks as Backbone, with a smaller size and lower computational cost. In our real-world scenarios, there is a need for object detection to faster inference. And YOLOv4-tiny [32] is a lightweight, fast object detection, which evolves from YOLOv3. YOLOv3 [33] consists of three parts. Darknet53 is the Backbone network of YOLOv3. Feature Pyramid Networks (FPN) [34] is feature fusion to strengthen the feature extraction network. YOLO head is the classifier and regressor to get the final result. YOLOv3 was improved to get YOLOv4 [35] and simplified to get YOLOv4-tiny.

3. Research framework and the proposed algorithm

In this section, the problem formulation and the framework of monitoring and counting by pieces are introduced. And the two core modules of the framework, the Feature Extractor based on YOLOv4-tiny and the Sliding Window Counter algorithm, are described.

¹ <https://github.com/GeLee-Q/Manual-Operations>

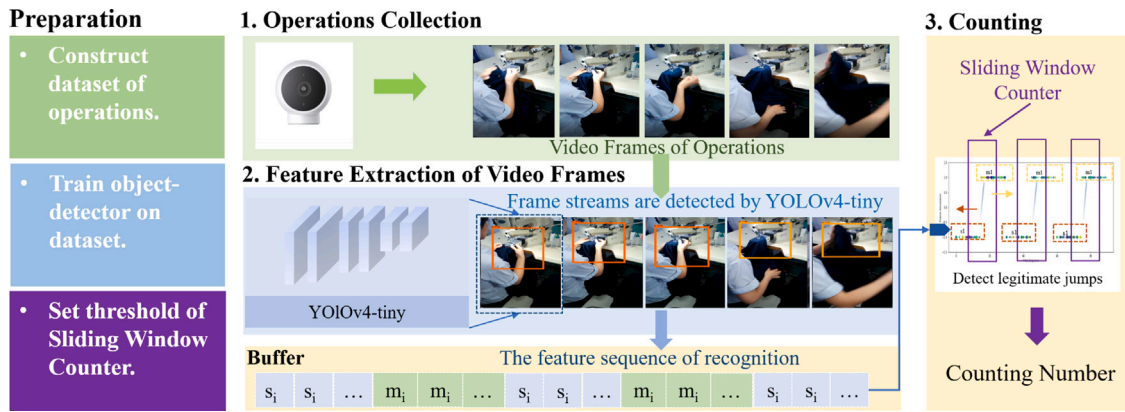


Fig. 1. Research framework.

3.1. Problem formulation

Workstation repetitive operation definition. Operators perform repetitive operations in the manufacturing workshop according to the standard operating procedure. The operation is composed of a sequence of one-actions.

In Eq. (1), it indicates that there are n repetitive operations. For tasks with repetition counting, identifying the first one-action and the last one-action in sequence ensures that one operation is completed, rather than trying to identify each one-action. In Eq. (2), the abstraction of *sub-operation* is introduced, and each operation consists of two *sub-operation*, which in turn consist of several one-actions. In Eqs. (3) and (4), f_{T_0} is the beginning frame of one operation, f_{T_2} is the ending frame of one operation. f_{T_1} is the key division frame that divides the operation into *sub-operation*₁ and *sub-operation*₂.

$$\text{Repetition} = \{\text{operation}_1, \text{operation}_2, \dots, \text{operation}_n\} \quad (1)$$

$$\text{operation} = \{\text{sub-operation}_1, \text{sub-operation}_2\} \quad (2)$$

$$\text{sub-operation}_1 = \{f_{T_0}, f_{T_0+1}, \dots, f_{T_1}\} \quad (3)$$

$$\text{sub-operation}_2 = \{f_{T_1+1}, f_{T_1+2}, \dots, f_{T_2}\} \quad (4)$$

Target formulation. Our goal is to identify and count operations to obtain real-time production data. The proposed Sliding Window Counter will check each division f_{T_1} of sub-operations whether is legitimate. The legitimate conditions of boundary are defined in Section 3.4.

$$\hat{O} = \begin{cases} 0, & f_{T_1} \text{ is not legitimate} \\ 1, & f_{T_1} \text{ is legitimate} \end{cases} \quad (5)$$

$$c = \sum_{i=1}^n \hat{O}_i \quad (6)$$

The $\hat{O}_i$ is 1 or 0 which denotes whether the boundary of *operation* _{i} is legitimate or not. The c denotes the counting results. The Eq. (6) shows that the counting results can be obtained by examining the video frames to find legitimate boundary of sub-operations.

3.2. Framework overview

Our framework aims to provide a complete process from dataset construction, model training, and repetitive operation counting. The preparation phase is shown on the left side of Fig. 1, the action sequences of operation are divided into two sub-operations, and the

video frames are labelled to build the dataset. Then a suitable object-detection is selected as the Feature Extractor and trained on the dataset to obtain the desired weights. Finally, considering the time scales of different operations, the Sliding Window Counter's threshold value is set.

The repetitive operation counting is shown on the right side of Fig. 1. At first, the camera captures the source video stream. Next, the video frame is processed by the Feature Extractor to locate the operation in the frames and get the class information of operations. The Feature Extractor and the following Sliding Window Counter algorithm are parallel processes and transfer data through the Buffer. The Feature Extractor continuously processes the video frames and feeds the results into the Buffer. The Sliding Window Counter then continuously reads the feature sequence from the Buffer, traverses the feature sequence, and gets the final count result.

3.3. The feature extractor based on YOLOv4-tiny

The Feature Extractor is utilized to extract the class information of the sub-operations in the video frame to encode the long-period operation. The detection result of object detection has both positioning information of the operation and class information of the sub-operations in the frame. Then the extracted class information of the sub-operations is put into the Buffer as the output of the Feature Extractor.

Several object detections have been selected to be the Feature Extractor. These algorithms are Faster R-CNN [26], SDD [29], EfficientDet [30], YOLOv3 [33], YOLOv4 [35] and YOLOv4-tiny [32]. Moreover, the effects of different algorithms are investigated in the experimental part. Since the high inference speed is essential to real-time production tracking, finally, YOLOv4-tiny is decided to use for the Feature Extractor part.

The overall structure of YOLOv4-tiny [32] is shown in Fig. 2. To sum up, the object detection is composed of several parts:

- **Input:** Image.
- **Backbone:** The backbone feature extraction network is CSPdarknet53-tiny. The backbone uses layer-by-layer convolutional networks to extract previously learned features. And the CSPNet [25] of backbone using a skipping connection reduces the number of parameters, increases the utilization of computation units, and eases memory footprint.
- **Neck:** As shown in Fig. 2, the Neck part adopts Feature Pyramid Networks (FPN) [34] structure, which uses a top-down process and horizontal connections for information fusion to avoid information loss due to network deepening.

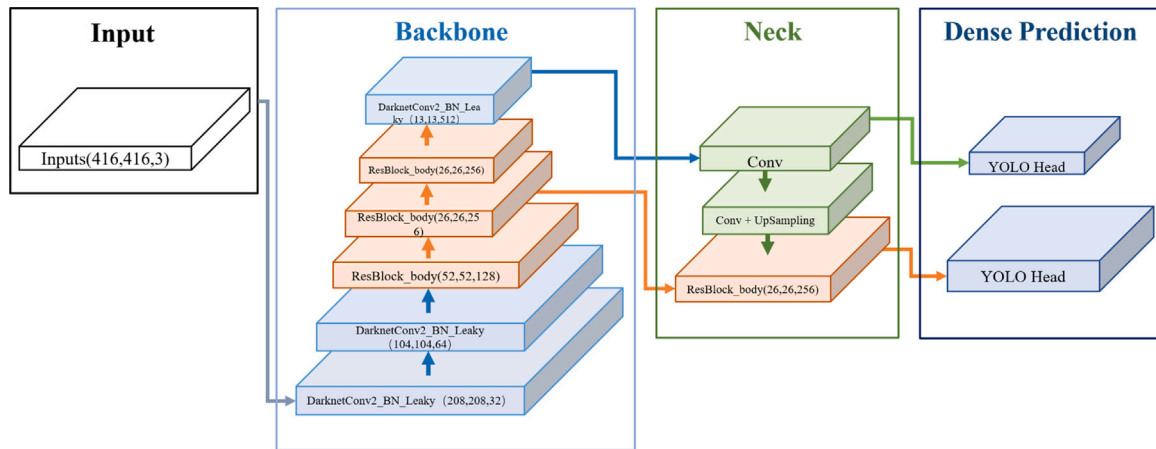
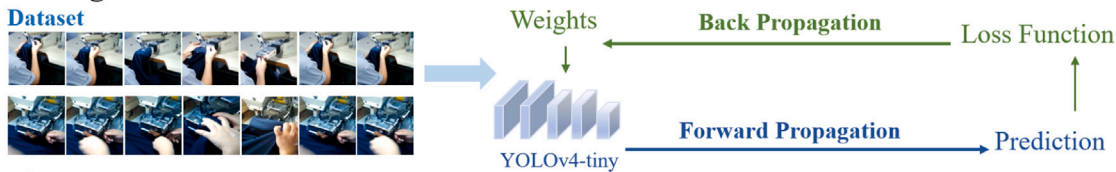


Fig. 2. The YOLOv4-tiny has three sections. (1) The role of the backbone is to extract the information from the image (2) Neck makes better use of the features extracted from the backbone. (3) Dense Prediction outputs the final result.

Training Process



Inference Process

The input video frames



The sequence of recognition

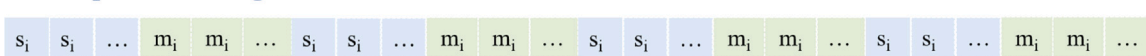


Fig. 3. The YOLOv4-tiny will train on the dataset to get the weight information of the relevant data. YOLOv4-tiny can utilize the trained weights to recognize the input video frames in the practice phase.

- **Dense Prediction:** The YOLO Head will predict the prediction boxes on the input image, which capture the manual operations and bring the category information of these operations.

In general, the loss function of YOLOv4-tiny consists of three parts:

$$\text{Loss(object)} = \text{loss(confidence)} + \text{loss(classification)} + \text{loss(regression)} \quad (7)$$

In Fig. 3, it can be seen that the object detection in the training procedure is forward propagation using the network weights, and the loss function is then back propagated to update the network weights using the prediction results from the forward propagation. In the inference procedure of practical application, YOLOv4-tiny directly uses the learned weights from the training procedure to compute the video frames to obtain the recognition results.

3.4. The sliding window counter algorithm

The two stages of Feature Extractor and the Sliding Window Counter are parallelly processed. The object detection algorithm in the first stage processes the video streams continuously. The extracted temporal information will be placed in a buffer. The second stage of the Sliding Window Counter is to get the data from the buffer and analyse it further. The data processed by Feature Extractor are put into the buffer continuously, and the Sliding Window Counter keeps processing and

counting. Thus, the proposed approach can count in real-time based on the parallel processes.

Each information in the sequence from the Feature Extractor represents the type of sub-operation to which the action in this frame belongs. This feature sequence is visualized by mapping this information into numeric values, with the first sub-operation represented by a negative value and the other sub-operation represented by a positive value. The Sliding Window Counter algorithm sets a window at each boundary of the positive and negative values, and this window will extend in both directions to collect state information. The Fig. 4 shows that the red box is the invalid counting box, while the purple box is the correct counting box. The principle of the algorithm and the basis for setting the window threshold are as follows.

The boundary between positive and negative values in the value sequence is a jump, representing the switch of this operation process from one sub-operation to another. However, many jumps in the sequence are noisy interferences, so a sliding window is set at the jump to collect information about the two sub-operations. The sliding window extended at the jump is legitimate if the following conditions are met.

- Condition 1: The number of continuous negative values in the sliding window to the left of the jump point is higher than the threshold.
- Condition 2: The number of continuous positive values in the sliding window to the right of the jump point is higher than the threshold.

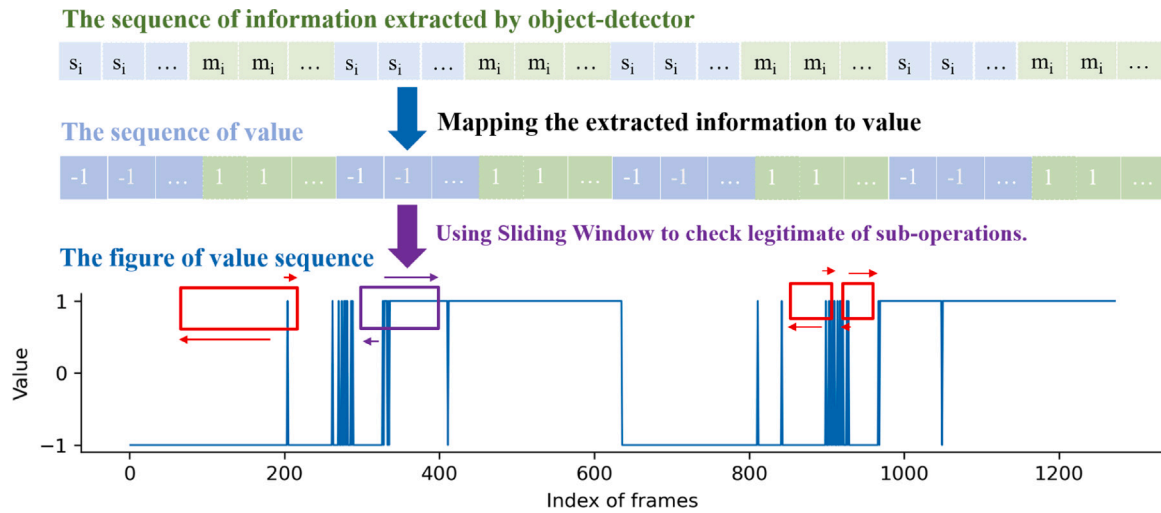


Fig. 4. The sliding window counter. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Algorithm 1: Sliding Window Counter

Input: Video detected sequence Q , the annotation sequence $Begin$ and Mid , the Threshold T .
Output: The number of the legitimate jump: Res .

```

1   $Last \leftarrow Q[0]$ 
2   $Cond1 \leftarrow False$ ,  $Cond2 \leftarrow False$ 
3   $Cond3 \leftarrow False$ 
4  for  $q$  in  $Q$  do
5      if  $q = Last$  then
6           $Cnt \leftarrow Cnt + 1$ 
7          if  $Last$  in  $Begin$  and  $Cnt < T$  then
8               $Cond1 \leftarrow True$ 
9          else if  $Last$  in  $Mid$  and  $Cnt < T$  then
10              $Cond2 \leftarrow True$ 
11      else
12           $Cnt \leftarrow 1$ 
13           $Last \leftarrow q$ 
14          if  $Cond3 = True$  and  $Cond2 = True$  then
15               $Res \leftarrow Res + 1$ 
16               $Cond3 \leftarrow False$ 
17               $Cond2 \leftarrow False$ 
18          end
19          if  $Cond1 = True$  then
20               $Cond3 \leftarrow True$ 
21               $Cond2 \leftarrow False$ 
22          end
23      end
24  end
25  return  $Res$ 

```

- Condition 3: Condition 1 needs to be satisfied before Condition 2 is satisfied.

The role of thresholds is to find legitimate jumps. The choice of the threshold value is related to video data processing. And the following are strategies for determining the range of thresholds: The number of video frames for one operation is N , and n frames are sampled for labelling. The overall content of n/N is used to represent the whole operation, reducing the computation and time during training. According to the sampling condition, the range of thresholds is defined:

$$\left[\frac{1}{3} * \frac{N}{n}, \frac{1}{2} * \frac{N}{n} \right] \quad (8)$$

Additionally, if the live video is down-sampled before the operation identifies and counts. The sampling rate is a :

$$a < \frac{N}{n} \quad (9)$$

Then Algorithm 1 is utilized to process the feature sequence, the input of Algorithm contains video frames, the sequences **Begin** and **Mid**. These two sequences contain all annotation information for the two sub-operations of the operation. The T denotes the eligible threshold. The Cnt indicates the number of frames of the current sub-operation phase that have occurred consistently. The **Cond1**, **Cond2** and **Cond3** are status flag variables indicating whether the window at the jump satisfies Condition 1, Condition 2 or Condition 3. In traversing the sequence, the previous jumps identified as noisy are automatically ignored by the sliding window of the next jump, and the counting result will be obtained at the end of the iteration.

4. Dataset: Repetitive manual operations

The largest video repetitive motion data currently available are *UCFRep* and *Countix*. The *UCFRep* by Zhang et al. [13] selected 23 categories from *UCF101* [36] in which the actions are performed cyclically. The dataset contains these categories, such as cutting, hammering, shaving, hula-hooping, soccer juggling, pedal jumping, cycling, ping pong shooting, handstand push-ups, rowing, etc. The period of these actions is between 0.12 s and 6.76 s. The *Countix* [12] is subset of the *Kinetics* [37] with segments of repeated actions and corresponding counts. The dataset contains 8757 videos in a total of 45 categories. *Countix* include repeated videos of workout activities (squats, pull-ups, battle rope training, exercising arm), dancing moves (pirouetting, pumping fist), playing instruments (playing ukulele), using tools repeatedly (hammer hitting objects, chainsaw cutting wood, slicing onion), artistic performances (hula-hooping, juggling soccer ball), sports (playing ping pong and tennis) and many others.

It is concluded that the actions of these datasets above are performing in people's daily lives. The characteristics of these actions are simple and strongly periodic, with cycle lengths below 3 s. However, the manual operations in the manufacturing environment are longer and more complex. Facing a more complex task, we used the object-detection dataset to construct our data. Each manual operation requires a fuzzy dichotomy of its one-actions sequence to redefine the sub-operations and allow the object detection to perform repetition counting. These processes are shown as follows.

Data collection. The datasets above are filtered from existing datasets, but the majority of our dataset is captured from manufacturing

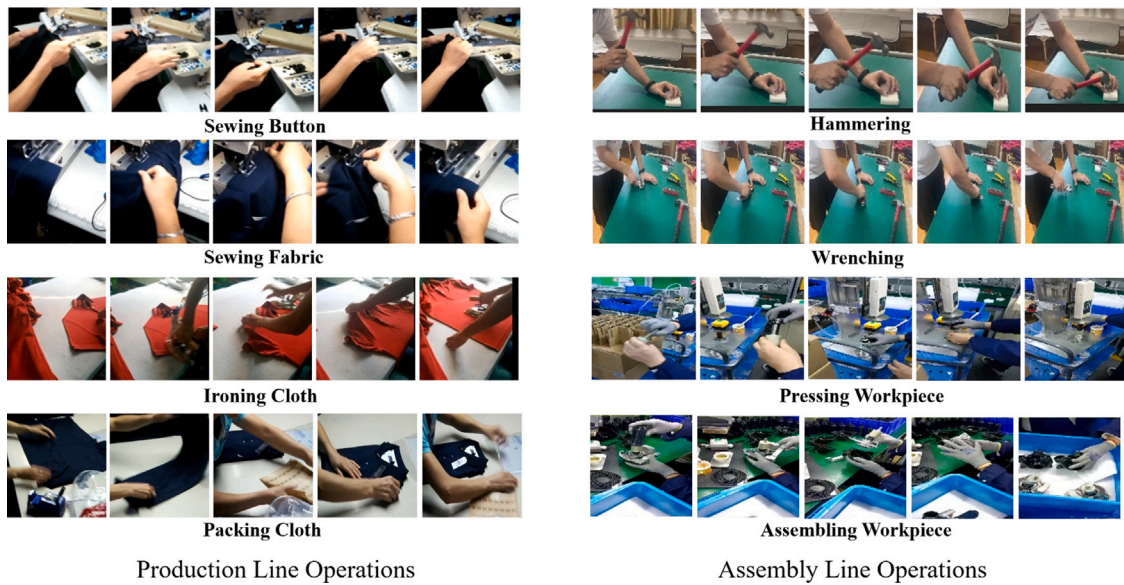


Fig. 5. Repetitive Manual Operation Dataset. In the left column, we present examples of repeating videos of production line operations, which are used to work out our method. In the right column, the two examples above are simple assembly operations, while the two examples below are complex manual assembly operations. These assembly operations are utilized to evaluate the generalization of our method.

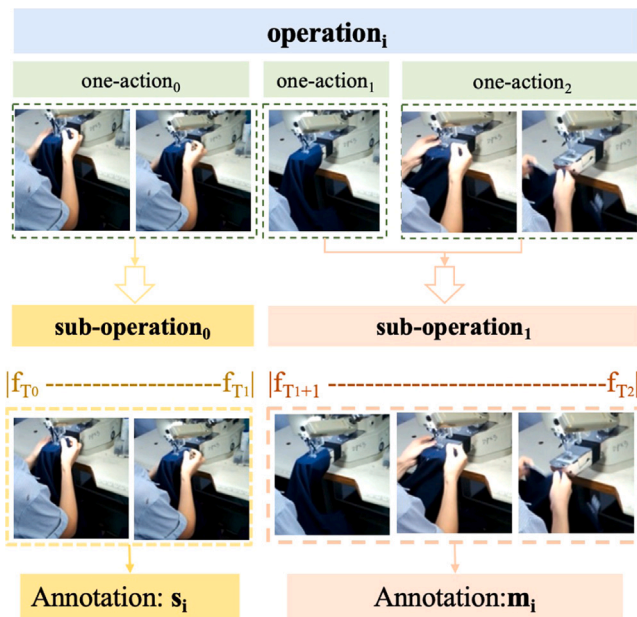


Fig. 6. The process of the ambiguous dichotomy of one-action sequences for one operation is shown. Each operation is divided into two sub-operations, and all frames in the specific sub-operations are labelled with the same annotation. For $operation_i$, during f_{T_0} to f_{T_1} , all frames are labelled as s_i , and other frames from f_{T_1+1} to f_{T_2} are labelled as m_i . s and m indicate the type of sub-operation. i indicates the type of operation, and i of different operations will be different values.

environment. In Fig. 5, the actions in the left column are operations of production line, collected from the garment factory, consisting of fabric spreading, marking, cutting, sewing, trimming, checking, finishing, pressing, and packing. The dataset of production line operation contains all the sewing processes to manufacture cloth, with 15 categories of manual operations that can be used for recognition and counting. Based on these actions, the experiments with different object-detections are performed to verify the effectiveness of our proposed method.

The ambiguous dichotomy of one-actions sequence. Each manual operation consists of a sequence of one-actions, and an operation

could be counted by identifying the first one-action and the last one-action. The information extraction of the one-actions between the first and the last can be processed vaguely instead of identifying each one-action precisely. Therefore, the concept of sub-operation is proposed to represent the hierarchy between operation and one-actions. Each operation consists of two sub-operations, and the sub-operation consists of several one-actions, thus enabling fuzzy dichotomous processing of one-actions sequence. This fuzzy division reduces the work required to annotate the data and allows object detection to converge to provide sufficiently accurate identification results. Furthermore, the method can be generalized to more complex manual operations.

$$\frac{1}{R} < \frac{T_2 - T_1}{T_1 - T_0} < R \quad (10)$$

In Fig. 6, the manual operation is composed of three one-actions. T_0 is the beginning of the operation, T_2 is the end of the operation, and the T_1 is the dividing point between the two sub-operations. The selection of the time point T_1 needs to take into account the length of time of the respective division parts. The Eq. (10) shows how to choose the T_1 point to divide one-action sequences. From the practice process, the value of R is generally set to 4.

Repetition labelling. Based on the ambiguous dichotomy of a sequence of one-actions, one operation can be divided into two sub-operations, and from that, each frame is labelled as ordered. Hundreds of frames are extracted from video, and these frames will be labelled into the format of the VOC dataset. Each frame's annotation has two layers of information: (1) The first level is the category of manual operations. (2) The second level is the category of sub-operations. In Fig. 6. The annotation of operation in video frames is shown.

Evaluation dataset. Additional assembly line operations are collected to explore the potential of the proposed method to deal with assembly line actions. Based on the research [8], three types of typical repetitive assembly actions are reconstructed; these actions are simple and short-cycle. Also, five types of long-period complex assembly actions processing workpieces are collected to validate that this method can handle complex manual assembly operations.

In Table 1, the period range of the operation for each data set is summarized. In our dataset of production line operation, the minimum cycle time is 2.5 s, while the maximum cycle time is 60 s. Our proposed method can handle repetitive counting of manual operations on clothing production lines. The assembly line action dataset is also

Table 1

The detail of dataset, the UCFRep [13], the Countix [12] and our dataset. The wide cycle length range between videos indicates that our task is more challenging than the previous study [12,13].

	Production line (Ours)	Assembly line (Ours)	UCFRep	Countix
Num. of Operations	15	8	23	45
Min of Cycles(s)	2.5	1.0	0.12	0.20
Max of Cycles(s)	60	70	6.76	7.69
The frames of Training Set	1700	715	–	–
The frames of Validation Set	200	90	–	–
The frames of Test Set	200	90	–	–

constructed as an evaluation dataset to verify that the method can be generalized to other complex manual scenarios. In the evaluation dataset, the shortest cycle time is 1 s, and the longest is the 70 s.

5. Experiment

5.1. Evaluation metric

The performance of counting of all methods are evaluated by same metrics are taken as the previous study [12–14]. The Mean Absolute Error (MAE) and Off-By-One (OBO) accuracy given K videos.

Mean Absolute Error (MAE) of count. The MAE aims to access counting errors directly, showing the mean of normalized absolute differences over the entire test videos.

$$MAE = \frac{1}{K} \sum_{i=1}^K \frac{|\hat{c}_i - c_i|}{c_i} \quad (11)$$

Off-By-One (OBO) count accuracy. OBO is the counting accuracy of all test videos.

$$OBO = \frac{1}{K} \sum_{i=1}^K [|\hat{c}_i - c_i| \leq 1] \quad (12)$$

Where $\hat{c}$ denotes the ground truth repetition counts and c_i denotes the detected repetition counts. The Eq. (12) shows that a count result with an error value above one would be considered a miss.

The performance of object detection on recognizing information of video frames are measured by following two metrics:

Mean Average Precision (MAP). In multiple category object detection, each category can draw a curve based on recall and precision, Average Precision (AP) is the area under that curve, and MAP is the average of multiple category AP. This value is between [0,1]. The larger, the better.

Frame Per Second (FPS). The metric indicates the detection speed of the model in one second. In order to collect real-time information in the manufacturing environment, inference speed is essential to the method.

5.2. Comparison with other methods

Comparison with state-of-the-art. The two techniques with the best generalization performance are applied to the captured videos. Debidatta et al. [12] proposed RepNet, which uses the temporal self-similarity matrix [38] as an intermediate layer to predict the cycle length and periodicity of each frame in the video. Zhang et al. [14] proposed Sound&Sight to solve the counting problem by vision and sound.

In Fig. 7, both methods have excessive counting results on most long-period and complex operations because both models are constructed based on simple and short-period actions. It is observed that when the operations are artificially accelerated by 4x in advance, the MAE of RepNet is reduced from 3.17 to 0.88, with the shorter the cycle, the better the performance. Based on the short-period property of the actions, RepNet would sample all input video frames down to 64 frames. For simple short-period actions, the sampling loss information is small, and the MAE value is low, but for long-period actions, the sampling loss information is extensive, and the MAE is high. In

addition to being based on short-period action, Sound & Sight relies more on the periodicity of sound and is sensitive to interference from high-frequency periodic noise. In real scenarios, the manufacturing machine generates irregular high-frequency short-period noise making the model appear to over-count. Compared with existing methods that would compress the input video information [12,14]. Our proposed method solves this problem in industrial scenes by using object detection to encode all captured frames with long-term information, avoiding information loss and interference from high-frequency noise.

5.3. Results and analysis

Alternative object-detections. We implement our object detection using PyTorch and train it with an NVIDIA Geforce2060 GPU. Our approach utilizes an object detection to extract information from the video frames and ultimately uses this information to realize counting. Therefore, for different object detections, both the speed and accuracy of their recognition of video frame information need to be considered. It is also necessary to examine the method's performance based on each object detection of repetitive counting.

In Fig. 8, the performance of different object detections for video frame recognition is presented. The two-stage object-detection, Faster R-CNN [26], has a MAP of 95.40% and FPS of 5. Faster R-CNN guarantees recognition accuracy with a complex process, an excessive number of parameters and redundant computation, but the excessive computation and memory usage drag down its speed of processing images. All the following object detections are one stage, simplifying the process of the two-stage model by making a trade-off between accuracy and speed, which has a significant advantage in FPS. The SSD [29] taking into account both advantages of SSD and YOLO, has the highest accuracy rate MAP of 95.73% and FPS of 25. EfficientDet [30] has MAP of 94.75% and FPS of 13. The low accuracy of this model may be due to fewer parameters, which makes it difficult to achieve better recognition. On the other hand, the YOLO series models have excellent performance in both aspects. YOLOv3 [33] has MAP of 95.66% and FPS of 16, which has the highest accuracy rate among the three models in the series. YOLOv4 [35] has MAP of 94.04% and FPS of 15. YOLOv4-tiny [32] has a MAP of 94.23% and FPS of 35. Its inference speed is substantially better than the other models. Compared to YOLOv3 and YOLOv4, YOLOv4 tiny decreases the complexity of the network architecture, computation and memory access, which leads to lower accuracy, but it is the fastest of all object detections.

The proposed method's performance based on each object detection with different thresholds is shown in Table 2. The information about the MAE of the method in Table 2 and the MAP of the object-detections in Fig. 8 is utilized to construct Fig. 9. In the case of the optimal threshold, in Fig. 9, the method with SSD, in which MAP is 95.73%, has the best performance of MAE (0.0428). While the method with YOLOv4, in which MAE is 94.04%, has the worst performance of MAE (0.1944). The method's performance based on different object-detections indicates that the accuracy of the information extracted from the video will affect the performance of the repeat counts. From the above results and analysis, it is concluded that the accuracy of the object detection in recognizing video frames is better, then the performance of the method based on such an object detection is also better in repetition counting.

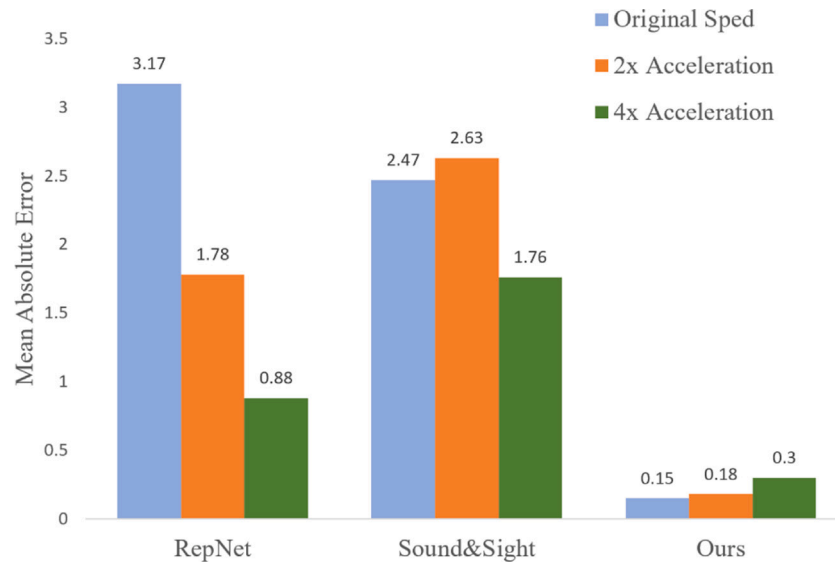


Fig. 7. Evaluation on the MAE of acceleration 1x, 2x and 4x on our dataset.

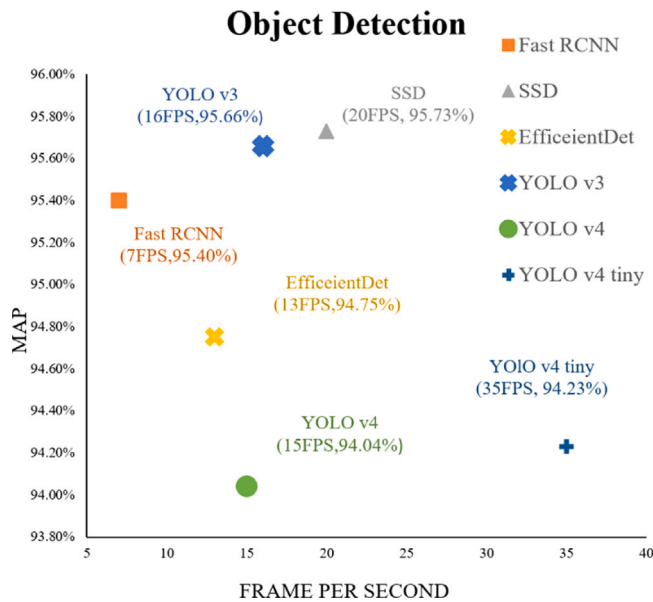


Fig. 8. The performance (MAP & FPS) of different Object Detections.

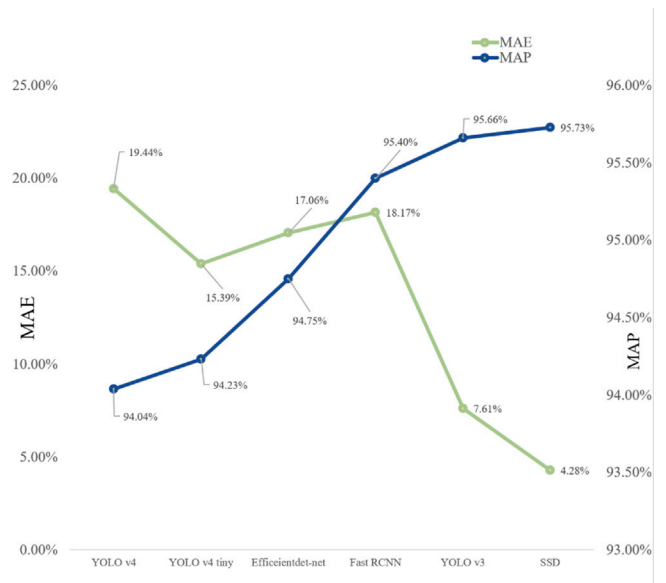


Fig. 9. The performance (MAE & MAP) of different Object Detections.

Considering that video frames captured by the camera in real scenes are at least 30FPS, a 30 FPS camera is chosen for cost-effectiveness. The proposed method needs to process video frames faster than 30 FPS to make real-time feedback. The YOLOv4-tiny is finally decided to use for Feature Extractor in method, which can get real-time recognition and counting number for production monitoring.

Sliding window counter. For the threshold value set by the Sliding Window Counter algorithm ensures to get the exact number of repetition operations. The noise and errors generated by the object detection during the feature extraction phase are filtered by setting the thresholds. A uniform distribution of thresholds is used according to Eq. (8) and the operation period to explore the effect of different thresholds on the method performance.

In Table 2, the threshold values for the best performance of the various object detection-based methods are also different. The method based on SSD performs well when the threshold value is set to 10. When using YOLOv4-Tiny, the algorithm performs well when the threshold

value is set to 20. As the threshold value increases or decreases gradually, the MAE value increases, and the model's performance worsens. The threshold stands for the sliding window size used to match the number of video frames represented by the duration of an operation. When the window is small, it will be disturbed by noise to produce a false positive, and when the window is large, it will contain multiple operations and carry out false counting results. Therefore, combined with the experiment results and the analysis of the Sliding Window Counter principle, the appropriate threshold value of the counter set could lead to better repetition counting.

The robustness to the timescale of our method. For further evaluating the robustness to the timescale of our method. We follow [22] to manually speed up the video to achieve different time scales. In addition to this, the experimentation of extending the period of operations has been done. When the video is processed at different speeds, it poses a challenge to the fixed timescale method [21]. As shown in Table 3, for operations under different acceleration conditions, by adjusting the range of thresholds, the value of MAE performs well. Since our method

Table 2

The performance of Our Method with different object detections and different thresholds.

Threshold value	Faster R-CNN		EfficientDet		SSD		YOLOv3		YOLOv4		YOLOv4-tiny	
	MAE	OBO	MAE	OBO	MAE	OBO	MAE	OBO	MAE	OBO	MAE	OBO
10	0.1984	0.86	0.1928	0.73	0.0428	0.86	0.0761	0.86	0.1984	0.80	0.2095	0.80
15	0.1984	0.86	0.1706	0.73	0.0650	0.86	0.0984	0.86	0.2206	0.73	0.2428	0.80
20	0.1817	0.73	0.1928	0.73	0.0650	0.86	0.0984	0.86	0.1944	0.80	0.1873	0.86
25	0.2317	0.73	0.2095	0.73	0.0817	0.86	0.0984	0.86	0.2595	0.66	0.1539	0.86
30	0.2317	0.73	0.2111	0.73	0.0722	0.86	0.1150	0.86	0.2880	0.60	0.1555	0.86
35	0.2888	0.66	0.2373	0.73	0.1484	0.80	0.1246	0.80	0.2976	0.60	0.2222	0.80
40	0.2984	0.66	0.3563	0.60	0.2007	0.73	0.2531	0.73	0.3309	0.60	0.2746	0.66
45	0.3269	0.60	0.4142	0.53	0.2492	0.66	0.2920	0.66	0.4238	0.53	0.3253	0.60
50	0.3682	0.53	0.4571	0.53	0.2587	0.66	0.3253	0.60	0.4333	0.53	0.3349	0.60
55	0.4444	0.40	0.4666	0.53	0.2777	0.66	0.3444	0.60	0.4333	0.53	0.3444	0.60
60	0.5111	0.40	0.5000	0.46	0.3444	0.60	0.3444	0.60	0.4333	0.53	0.3444	0.60

Table 3

The robustness to time-scale of our method.

Acceleration	YOLOv4-tiny	
	Threshold	MAE
0.5×	20	0.176
	40	0.187
	60	0.187
1.0×	20	0.187
	25	0.154
	30	0.155
2.0×	10	0.197
	15	0.220
	20	0.287
4.0×	5	0.187
	10	0.303
	15	0.466

Table 4

Evaluation on operations of assembly line.

Threshold value	YOLOv4-tiny	
	MAE	OBO
5	0.375	0.5
10	0.1875	0.375
15	0.3	0.875
20	0.325	0.75
25	0.35	0.75
30	0.45	0.625
35	0.5	0.625
40	0.5	0.625

does not sample the input video during the execution phase, there is no loss of information that could affect the counting. And by adjusting the threshold value of the Sliding Window Counter, the method can be adapted to different time scales of operation. Therefore, our method is robust to speed variations, detecting the repetitions with different time scales.

5.4. Evaluation on operations of assembly line

Our proposed method to count repetitive wild manual operations requires the following processing steps. Each operation should be divided into two sub-operations and construct the corresponding dataset according to the process we mentioned earlier. The object detection is then trained on the newly constructed dataset. Finally, setting the appropriate threshold for the Sliding Window Counter will give the result of the repetition counting.

In Table 4, we compare the method based on YOLOv4-tiny with the different threshold values. Its best performance is similar to the proposed method's performance on production line operations. Experimental results on the evaluation dataset indicate that our method could effectively count repetitions on other simple and complex manual operation scenarios.

It is a promising research practice for deep learning methods in intelligent manufacturing. In Fig. 10, the entire whole application flow of the method for different manual operation scenarios. The top three are clothing production line scenarios and the bottom three are assembly line operation scenarios. Even if the period of the manual operation reaches long, our strategy of using an object detection to encode ultra-long-range information in the video is effective. Compared to the end-to-end approach of training and processing the entire video, lightweight object detection saves the cost and time of the training and inference process. Our method provides a cost-effective solution for deployment in manufacturing workshops.

6. Conclusion

In this paper, a novel framework for monitoring manual operations of production/assembly lines with machine vision is presented. And two-stage approach to counting by pieces in real time for manual operations of production/assembly lines is put forward. This method provides a new and effective method for the real-time perception of workpiece ratios of manually operated production/assembly lines, which makes it possible for their smart and refinement management. In the two-stage approach, the YOLOv4-tiny is used to detect workers' sub-operations for each captured video frame and form the feature sequence, and the Sliding Window Counter is presented to search the mutation signals the feature sequence in real-time and record the numbers of mutation signals. Experiments show that the method is validated to monitor and count by pieces for production/assembly lines, the FPS reaches 35, and MAE is 0.1593. The future work of this research is to explore general monitoring and counting method with machine vision for manually operated production/assembly lines. It can not only count by pieces more accurately according to the video monitoring information but also evaluate the standardization and rationality of manually operated production/assembly lines.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported in part by the National Natural Science Foundation of China (Grant No. 52075404), and in part by the Application Foundation Frontier Special Project of Wuhan Science and Technology Bureau (Grant No. 2020010601012176).

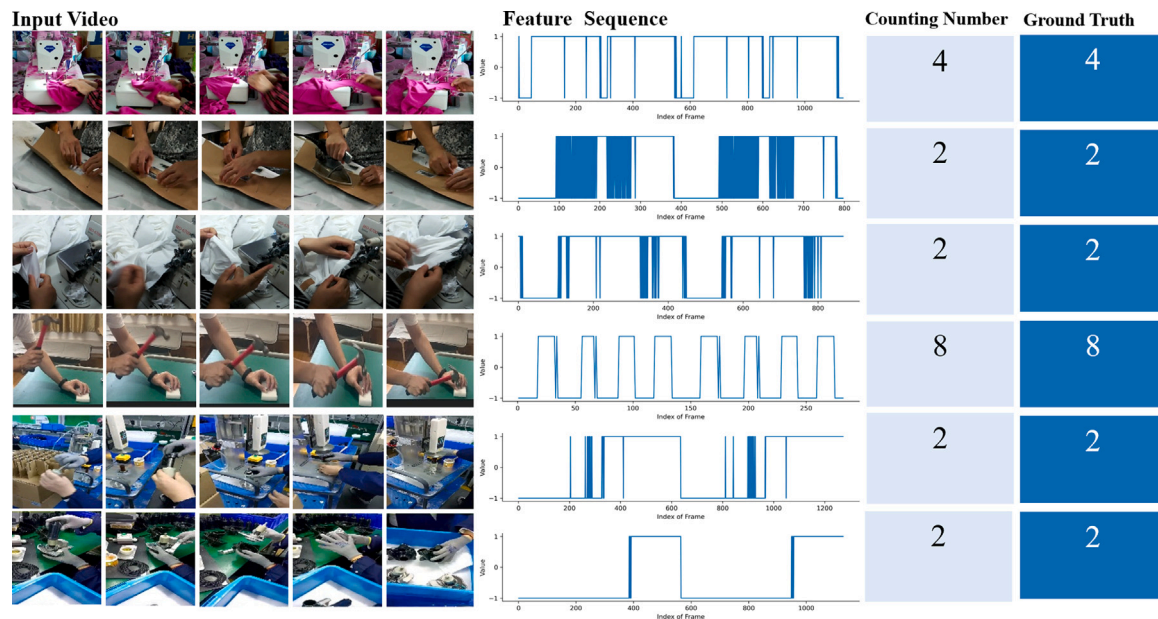


Fig. 10. The results' visualization of different manual operations.

References

- [1] Xu X, Lu Y, Vogel-Heuser B, Wang L. Industry 4.0 and Industry 5.0-Inception, conception and perception. *J Manuf Syst* 2021;61:530–5. <http://dx.doi.org/10.1016/j.jmsy.2021.10.006>, URL <https://www.sciencedirect.com/science/article/pii/S0278612521002119>.
- [2] Wang J, Ma Y, Zhang L, Gao RX, Wu D. Deep learning for smart manufacturing: Methods and applications. *J Manuf Syst* 2018;48:144–56. <http://dx.doi.org/10.1016/j.jmsy.2018.01.003>, URL <https://www.sciencedirect.com/science/article/pii/S0278612518300037> [special issue on smart manufacturing].
- [3] Lai X, Shui H, Ding D, Ni J. Data-driven dynamic bottleneck detection in complex manufacturing systems. *J Manuf Syst* 2021;60:662–75. <http://dx.doi.org/10.1016/j.jmsy.2021.07.016>, URL <https://www.sciencedirect.com/science/article/pii/S0278612521001539>.
- [4] Bian S, Li C, Fu Y, Ren Y, Wu T, Li G-P, et al. Machine learning-based real-time monitoring system for smart connected worker to improve energy efficiency. *J Manuf Syst* 2021;61:66–76. <http://dx.doi.org/10.1016/j.jmsy.2021.08.009>, URL <https://www.sciencedirect.com/science/article/pii/S0278612521001813>.
- [5] Kim J, Lee H, Jeong S, Ahn S-H. Sound-based remote real-time multi-device operational monitoring system using a convolutional neural network (CNN). *J Manuf Syst* 2021;58:431–41. <http://dx.doi.org/10.1016/j.jmsy.2020.12.020>, URL <https://www.sciencedirect.com/science/article/pii/S027861252030234X>.
- [6] Kao Y-T, Chang S-C. Setting daily production targets with novel approximation of target tracking operations for semiconductor manufacturing. *J Manuf Syst* 2018;49:107–20. <http://dx.doi.org/10.1016/j.jmsy.2018.09.003>, URL <https://www.sciencedirect.com/science/article/pii/S0278612518303480>.
- [7] Tao F, Qi Q, Liu A, Kusiak A. Data-driven smart manufacturing. *J Manuf Syst* 2018;48:157–69.
- [8] Chen C, Wang T, Li D, Hong J. Repetitive assembly action recognition based on object detection and pose estimation. *J Manuf Syst* 2020;55:325–33. <http://dx.doi.org/10.1016/j.jmsy.2020.04.018>, URL <https://www.sciencedirect.com/science/article/pii/S0278612520300625>.
- [9] Wei S-E, Ramakrishna V, Kanade T, Sheikh Y. Convolutional pose machines. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, p. 4724–32.
- [10] Xiong Q, Zhang J, Wang P, Liu D, Gao RX. Transferable two-stream convolutional neural network for human action recognition. *J Manuf Syst* 2020;56:605–14. <http://dx.doi.org/10.1016/j.jmsy.2020.04.007>, URL <https://www.sciencedirect.com/science/article/pii/S0278612520300510>.
- [11] Tao W, Lai Z-H, Leu MC, Yin Z. Worker activity recognition in smart manufacturing using IMU and semg signals with convolutional neural networks. *Procedia Manuf* 2018;26:1159–66. <http://dx.doi.org/10.1016/j.promfg.2018.07.152>, URL <https://www.sciencedirect.com/science/article/pii/S235197891830828X>, 46th SME North American Manufacturing Research Conference, NAMRC 46, Texas, USA.
- [12] Dwibedi D, Aytar Y, Tompson J, Sermanet P, Zisserman A. Counting out time: Class agnostic video repetition counting in the wild. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020.
- [13] Zhang H, Xu X, Han G, He S. Context-aware and scale-insensitive temporal repetition counting. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, p. 670–8.
- [14] Zhang Y, Shao L, Snoek CGM. Repetitive activity counting by sight and sound. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, p. 14070–9.
- [15] Hussein N, Gavves E, Smeulders AWM. Timeception for complex action recognition. In: *CVPR*. 2019.
- [16] Runia TFH, Snoek CGM, Smeulders AWM. Repetition estimation. *Int J Comput Vis* 2019;127:1361–83.
- [17] Soro A, Brunner G, Tanner S, Wattenhofer R. Recognition and repetition counting for complex physical exercises with deep learning. *Sensors* 2019;19:714.
- [18] Cutler R, Davis LS. Robust real-time periodic motion detection, analysis, and applications. *IEEE Trans Pattern Anal Mach Intell* 2000;22:781–96. <http://dx.doi.org/10.1109/34.868681>, URL <http://ieeexplore.ieee.org/document/868681/>.
- [19] Azy O, Ahuja N. Segmentation of periodically moving objects. In: *2008 19th International Conference on Pattern Recognition*. Tampa, FL, USA: IEEE; 2008, p. 1–4. <http://dx.doi.org/10.1109/ICPR.2008.4760949>, URL <http://ieeexplore.ieee.org/document/4760949/>.
- [20] Pogalin E, Smeulders AWM, Thean AHC. Visual quasi-periodicity. In: *2008 IEEE Conference on Computer Vision and Pattern Recognition*. Anchorage, AK, USA: IEEE; 2008, p. 1–8. <http://dx.doi.org/10.1109/CVPR.2008.4587509>, URL <http://ieeexplore.ieee.org/document/4587509/>.
- [21] Levy O, Wolf L. Live repetition counting. In: *2015 IEEE International Conference on Computer Vision*. Santiago, Chile: IEEE; 2015, p. 3020–8. <http://dx.doi.org/10.1109/ICCV.2015.346>, URL <http://ieeexplore.ieee.org/document/7410703/>.
- [22] Runia TFH, Snoek CGM, Smeulders AWM. Real-world repetition estimation by div, grad and curl. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, p. 9009–17.
- [23] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition. 2015, arXiv:1409.1556.
- [24] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, p. 770–8.
- [25] Wang C-Y, Liao H-YM, Wu Y-H, Chen P-Y, Hsieh J-W, Yeh I-H. Cspnet: A new backbone that can enhance learning capability of CNN. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*. 2020, p. 390–1.
- [26] Ren S, He K, Girshick R, Sun J. Faster r-cnn: Towards real-time object detection with region proposal networks. *Adv Neural Inf Process Syst* 2015;28:91–9.
- [27] Girshick R. Fast R-CNN. In: *Proceedings of the IEEE international conference on computer vision*. 2015.
- [28] Redmon J, Divvala S, Girshick R, Farhadi A. You only look once: Unified, real-time object detection. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, p. 779–88.
- [29] Liu W, Anguelov D, Erhan D, Szegedy C, Reed S, Fu C-Y, et al. Ssd: Single shot multibox detector. In: *European conference on computer vision*. Springer; 2016, p. 21–37.

- [30] Tan M, Pang R, Le QV. Efficientdet: Scalable and efficient object detection. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2020, p. 10781–90.
- [31] Tan M, Le Q. Efficientnet: Rethinking model scaling for convolutional neural networks. In: International conference on machine learning. PMLR; 2019, p. 6105–14.
- [32] Wang C-Y, Bochkovskiy A, Liao H-YM. Scaled-yolov4: Scaling cross stage partial network. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2021, p. 13029–38.
- [33] Redmon J, Farhadi A. Yolov3: An incremental improvement. 2018, arXiv preprint [arXiv:1804.02767](https://arxiv.org/abs/1804.02767).
- [34] Lin T-Y, Dollár P, Girshick R, He K, Hariharan B, Belongie S. Feature pyramid networks for object detection. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2017, p. 2117–25.
- [35] Bochkovskiy A, Wang C-Y, Liao H-YM. Yolov4: Optimal speed and accuracy of object detection. 2020, arXiv preprint [arXiv:2004.10934](https://arxiv.org/abs/2004.10934).
- [36] Soomro K, Zamir AR, Shah M. UCF101: A dataset of 101 human actions classes from videos in the wild. 2012, arXiv preprint [arXiv:1212.0402](https://arxiv.org/abs/1212.0402).
- [37] Kay W, Carreira J, Simonyan K, Zhang B, Hillier C, Vijayanarasimhan S, et al. The kinetics human action video dataset. 2017, arXiv preprint [arXiv:1705.06950](https://arxiv.org/abs/1705.06950).
- [38] Panagiotakis C, Karvounas G, Argyros A. Unsupervised detection of periodic segments in videos. In: 2018 25th IEEE international conference on image processing. IEEE; 2018, p. 923–7.